UNIVERSITY OF ASIA PACIFIC

Department of Computer Science and Engineering

PROJECT REPORT

Design and Simulation of a Mini 8-bit Accumulator CPU Using VHDL

Course: CSE 450 – VLSI Lab
Project: Mini 8-bit CPU – Assigned Set 0

Item	Details
Student Name	Ankita Hossain Anushka
Student ID	22201130
Program / Department	CSE, University of Asia Pacific
Section	C (update if required)
Course Instructor	Md. Rasheduzzaman
Assigned Set	Set 0 (last digit of Student ID = 0)
Simulation Tool	GHDL, VHDL-2008
Waveform Viewer	GTKWave
Submission Date	29.10.2026

Abstract

This project presents the design and simulation of a small 8-bit accumulator-based central processing unit (CPU) using VHDL. The processor implements five instructions: Load Immediate (LDI), Add (ADD), bitwise AND (AND), conditional Jump if Zero (JZ), and bitwise NOT (NOT). The architecture contains a 4-bit program counter, a 16 × 12-bit program ROM, a combinational control unit, an 8-bit arithmetic logic unit (ALU), and an accumulator with a zero flag. The CPU follows a single-cycle design in which instruction fetch, decode, and execution occur within one rising clock edge. A self-checking VHDL

testbench verifies the expected accumulator, zero-flag, and program-counter values. Simulation completed successfully: all seven instruction cycles matched the expected results.

1. Introduction

A CPU executes instructions by fetching them from program memory, decoding the opcode, performing the required operation, and updating its internal state. This project implements a simplified accumulator CPU to demonstrate the basic datapath and control concepts using VHDL. The accumulator is the main working register, while the zero (Z) flag indicates whether the most recent ALU result is zero.

2. Objectives

- Design a functional 8-bit accumulator-based CPU using VHDL.
- Implement the assigned instruction set: LDI, ADD, AND, JZ, and NOT.
- Construct the ALU and supporting logic using basic logic-gate operations.
- Implement program sequencing with a 4-bit program counter and conditional branching.
- Verify CPU operation using a self-checking testbench and waveform analysis.

3. Tools and Technologies

Tool / Technology	Purpose
VHDL (IEEE std_logic_1164, numeric_std)	Hardware description and simulation model
GHDL with --std=08	VHDL analysis, elaboration, and simulation
GTKWave	Viewing signal waveforms
Windows batch script (run_cpu.bat)	Automated compilation and simulation

4. Assigned Instruction Set

Student ID 22201130 ends in 0; therefore, the assigned configuration is Set 0. The mandatory instructions are LDI and ADD, and the additional assigned instructions are AND, JZ, and NOT.

Instruction	Opcode	Operation	Effect
LDI	0001	Load immediate	ACC ← operand
ADD	0010	Addition	ACC ← ACC + operand (8-bit result)
AND	0100	Bitwise AND	ACC ← ACC AND operand
NOT	0111	Bitwise complement	ACC ← NOT ACC
JZ	1100	Jump if zero	If Z=1, PC ← operand[3:0]; otherwise PC increments
NOP	0000 / other unsupported	No operation	No register write; PC increments

5. CPU Architecture and Design

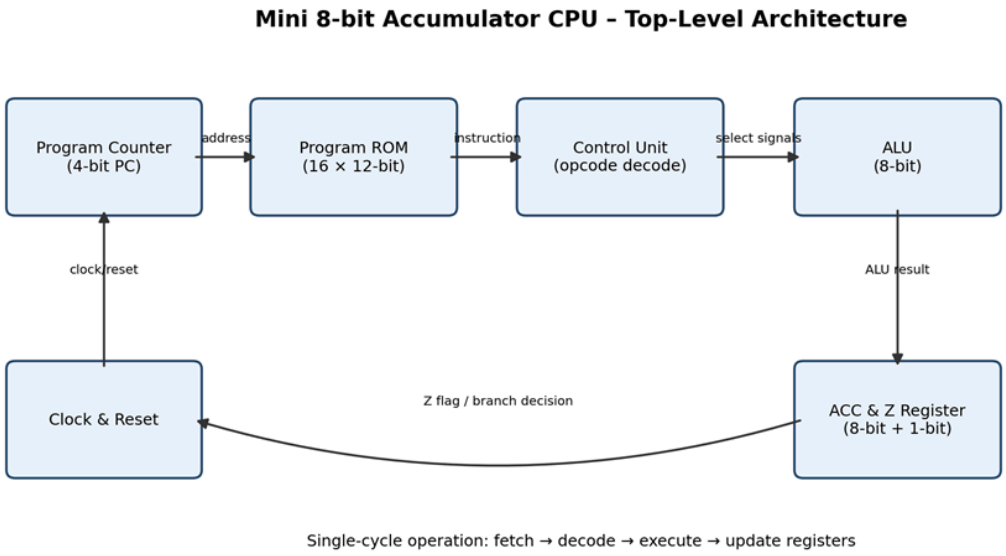


Figure 1. Top-level architecture of the mini 8-bit CPU.

5.1 Program Counter (PC)

The program counter is a 4-bit register that addresses up to 16 instruction locations. Under normal operation it increments by one each cycle. When the decoded instruction is JZ and the current Z flag is 1, the PC loads the lower four bits of the instruction operand. Reset sets the PC to 0.

5.2 Program ROM

The program ROM stores sixteen 12-bit instructions. Each instruction contains a 4-bit opcode and an 8-bit operand. The implemented program is:

Address	Machine Code	Instruction
0	1A5	LDI 0xA5
1	45A	AND 0x5A
2	C05	JZ 0x05
3	201	ADD 0x01
4	201	ADD 0x01
5	700	NOT
6	40F	AND 0x0F
7	C00	JZ 0x00
8	201	ADD 0x01
9–15	000	NOP / unused

5.3 Control Unit

The control unit decodes the 4-bit opcode using Boolean logic. It generates ALU operation-select signals, accumulator write enable (acc_we), zero-flag write enable (z_we), and the JZ control signal. For the four ALU instructions, ACC and Z are updated. JZ affects only the PC.

5.4 Arithmetic Logic Unit (ALU)

The 8-bit combinational ALU supports immediate loading, addition, bitwise AND, and bitwise NOT. Addition uses an 8-bit ripple-carry adder assembled from full-adder stages. The carry-out is not stored, so the result is limited to eight bits. A NAND-based one-hot multiplexer selects the required operation. Two 4-input NOR stages and an AND gate generate the zero flag.

5.5 Accumulator and Zero Flag

The accumulator stores the 8-bit ALU result. The Z register stores 1 when the selected ALU result is 0x00 and 0 otherwise. Both registers use a synchronous, active-high reset. On reset, ACC becomes 0x00 and Z becomes 1.

6. Instruction Execution and Expected Results

The testbench holds reset active for the first rising edge, then checks seven instruction cycles. The JZ at address 2 is taken because AND produces zero, so execution jumps to address 5 and skips addresses 3 and 4. The JZ at address 7 is not taken because the zero flag is 0.

Cycle	Executed instruction	ACC (hex)	Z	PC after cycle
0	RESET	00	1	0
1	LDI 0xA5	A5	0	1
2	AND 0x5A	00	1	2
3	JZ 0x05 (taken)	00	1	5
4	NOT	FF	0	6
5	AND 0x0F	0F	0	7
6	JZ 0x00 (not taken)	0F	0	8
7	ADD 0x01	10	0	9

7. Simulation Procedure

1. Place all VHDL source files in one project directory.
2. Open a terminal in that directory and ensure the GHDL executable is available in PATH.
3. Compile all design and testbench files using VHDL-2008.
4. Elaborate the testbench entity.

5. Run the simulation and generate the GHW waveform file.
6. Open cpu.ghw in GTKWave and inspect clk, reset, pc, q (ACC), and z.

Commands (also automated by run_cpu.bat):

ghdl -a --std=08 gates.vhd alu.vhd control_unit.vhd program_rom.vhd pc.vhd acc_z_reg.vhd cpu_top.vhd tb_cpu.vhd
ghdl -e --std=08 tb_cpu
ghdl -r --std=08 tb_cpu --wave=cpu.ghw

8. Simulation Results

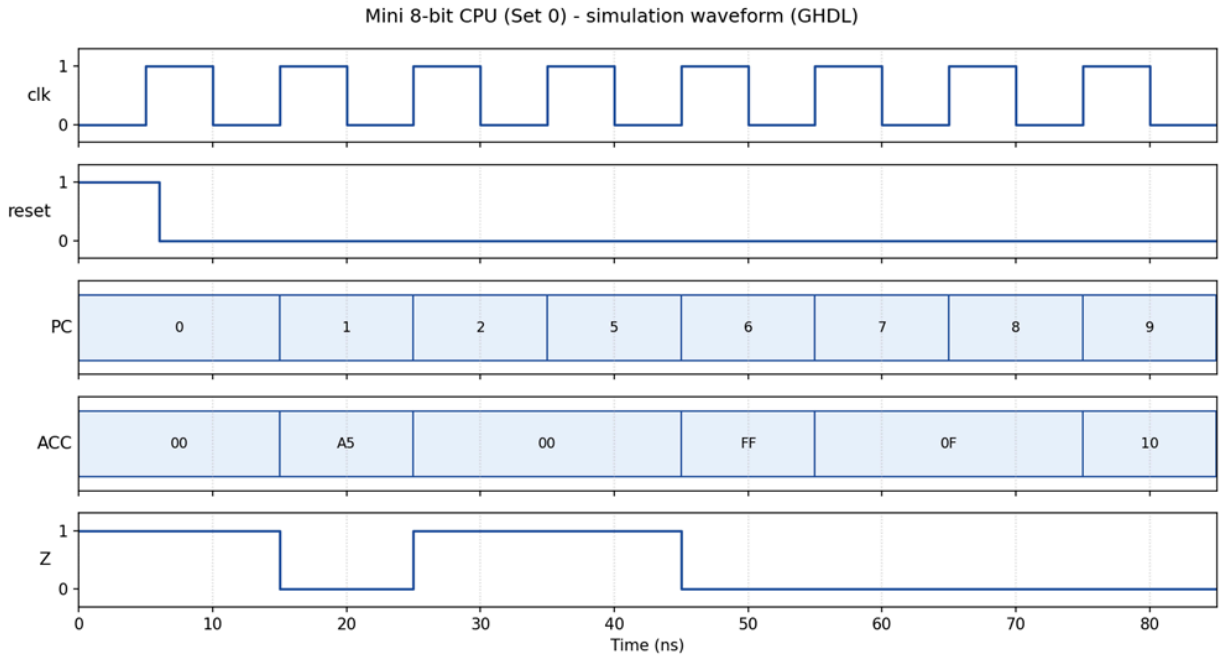


Figure 2. Simulation waveform showing clock, reset, PC, accumulator, and zero flag.

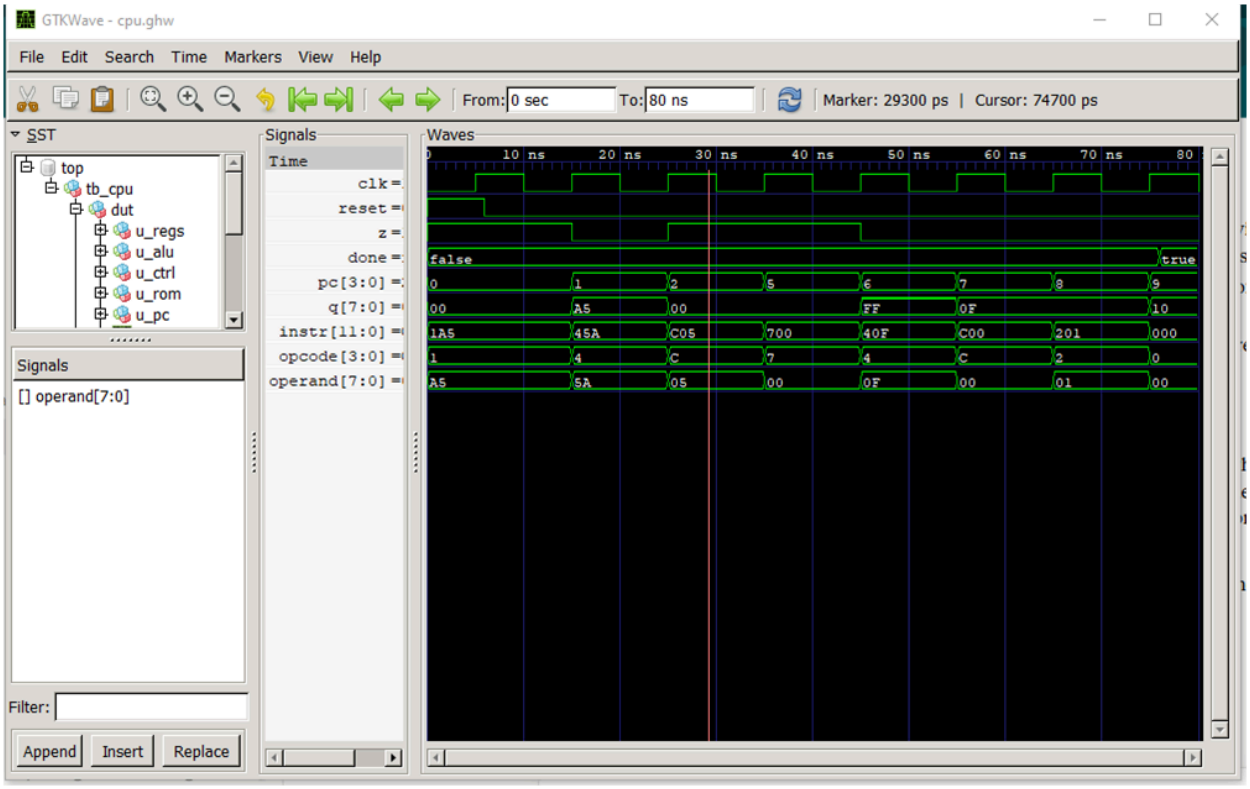


Figure 3. GTKWave simulation view (provided project screenshot).

The simulation log reports Cycle 1 OK through Cycle 7 OK, followed by: “PASS: all cycles match Appendix B (Set 0).” This confirms that the observed ACC, Z, and PC values match the testbench expectations for every checked cycle. A numeric_std warning at time 0 indicates that the ROM address briefly contains an uninitialized value before reset takes effect; it is a startup warning, and the subsequent checked cycles pass.

9. Result and Discussion

The mini CPU successfully executed the assigned Set 0 instruction sequence. LDI loaded 0xA5 into the accumulator. AND with 0x5A generated 0x00 and asserted the zero flag. The first JZ therefore redirected execution to address 5. NOT changed 0x00 to 0xFF, AND 0x0F produced 0x0F, and the second JZ was not taken because Z was 0. The final ADD produced 0x10. The self-checking testbench detected no cycle mismatches.

10. Conclusion

The project demonstrates a working single-cycle 8-bit accumulator CPU implemented in VHDL. Its datapath, control logic, program memory, conditional branch mechanism, and status flag were verified through GHDL simulation. All seven instruction cycles passed the expected-value checks. Future

improvements may include additional instructions, a larger register file, memory read/write operations, and a pipelined architecture.

11. Project Files

File	Description
cpu_top.vhd	Top-level CPU interconnection
gates.vhd	Full adder, 8-bit ripple-carry adder, and 4-bit incrementer
alu.vhd	8-bit ALU and zero detection
control_unit.vhd	Opcode decoder and control-signal generation
program_rom.vhd	16 × 12-bit instruction ROM
pc.vhd	4-bit program counter and JZ branch selection
acc_z_reg.vhd	Accumulator and zero-flag registers
tb_cpu.vhd	Self-checking testbench
run_cpu.bat	Compile, elaborate, and simulate script
cpu.ghw	Generated GHDL waveform file
sim_log.txt	Simulation output and PASS log
waveform.png / gtkwave_screenshot.png	Waveform evidence

12. GitHub

Git repository link : https://github.com/ankitahossain/8bit_CPU_Final_Project

13. References

1. IEEE, IEEE Standard VHDL Language Reference Manual (IEEE Std 1076).

2. GHDL User and Reference Manual, https://ghdl.github.io/ghdl/

3. GTKWave documentation, https://gtkwave.sourceforge.net/

4. Project source files and simulation log included in the submitted 8-bit CPU project archive.